Bringing Up the Cortex-M4 on the OMAP5432

Ducati/IPU architecture, Linux remoteproc, memory translation, safe boot, and the proven Phase 9.12 firmware

Practical engineering guide · DragonBox Pyra · 24 October 2026
Target: TI OMAP5432; Debian 12 / Linux 5.6.19-pyradef-m4test; internal SRAM experiment
Audience: Linux developers and embedded engineers who are new to OMAP5's auxiliary processors

---

The central idea
The Cortex-A15 running Linux does not simply jump into an M4 program. Linux must first enable the IPU device, place the firmware in the memory the IPU can use, prepare reset and cache/MMU state, and then release the intended M4 core. The **host firmware-loading address**, the **M4-local execution address**, and the **physical SRAM address** are different descriptions of the same bytes. The experiment succeeds only when all three agree.

What is proven on this particular Pyra: Phase 9.11.5 booted the small v5 program and produced `M5A1`, `M5B2`, `M5D3`. Phase 9.12 ran freestanding C from the internal SRAM, completed its checks, and produced four increasing heartbeat samples: `0x02D1`, `0x02DC`, `0x02E6`, `0x02F1`. This validates a specific CPU0-only, local-SRAM configuration—not all possible IPU memory, cache, or IPC configurations. [3–5]

A key safety boundary: This is a report of a tested configuration, **not** permission to write arbitrary MMU or PRM registers. Hardware initialization remains board-, firmware-, and kernel-version-dependent.

1. The hardware in plain English

1.1 Two different kinds of processor in one chip

The OMAP5432 contains two ARM Cortex-A15 application processors (the **MPU**) and an **IPU** subsystem with two ARM Cortex-M4 microcontroller cores. The Cortex-A15s run the DragonBox Pyra's Linux system and normal applications. The Cortex-M4 cores are separate processors, with their own reset controls and firmware. The operating system running on the A15 does not automatically execute on an M4.

The IPU cores can perform small real-time or control workloads while the A15s do other work. They share IPU internal memory and a shared cache subsystem, but that does not mean they behave like two normal Linux SMP cores. In our experiments Linux owns the startup process, and only **IPU CPU0** is released from reset; **CPU1 is deliberately held in reset**. The Texas Instruments technical reference manual (TRM) explicitly describes one-core and two-core operating modes. [1]

1.2 What do "Ducati" and "IPU" mean?

Ducati is Texas Instruments' historical software/architecture family name for a dual-core ARM Cortex-M subsystem. You may find it in older SYS/BIOS packages, example projects, software directory names, or discussions of OMAP4 and OMAP5. It is not an additional processor, a separate chip, or a firmware file format. Some older Ducati material describes **dual Cortex-M3**; the OMAP5432 IPU uses **dual Cortex-M4**. Use the OMAP5432 device documentation—not an OMAP4 example—as the authority for registers and reset behavior. [1, 8]

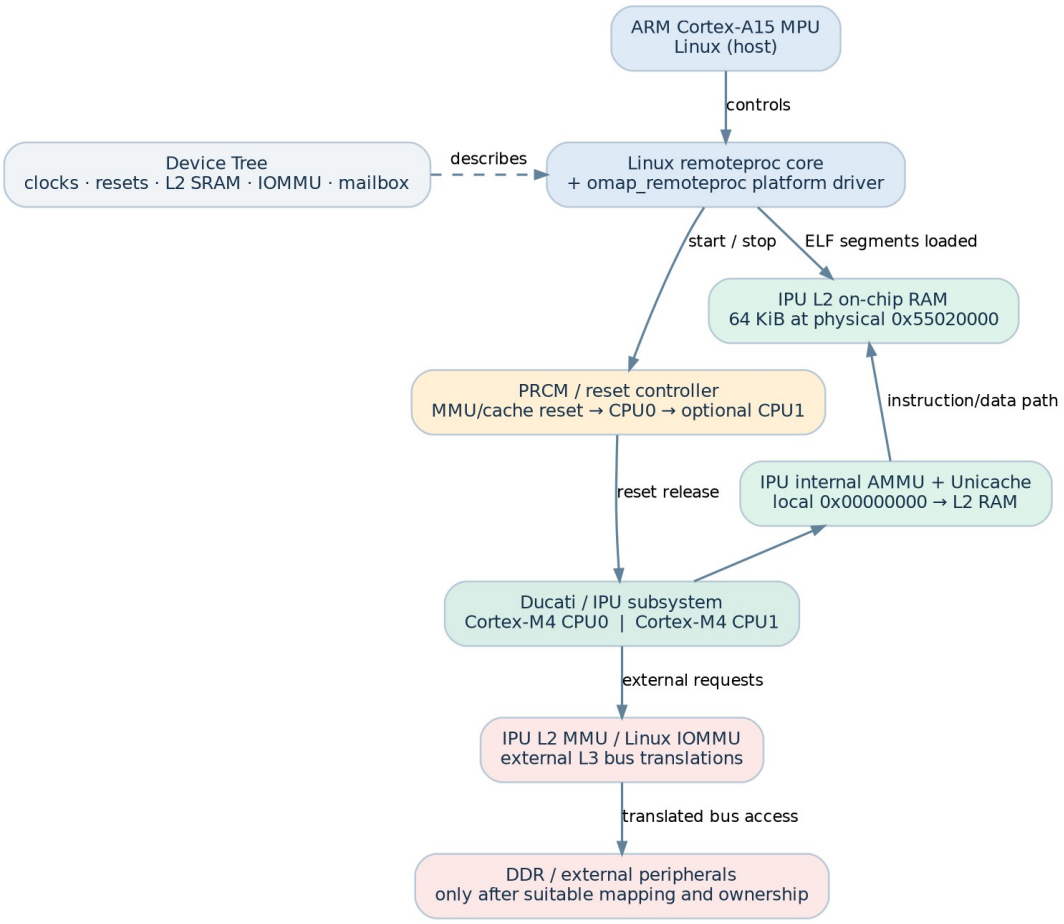
IPU, in this guide, means the OMAP5432 *dual Cortex-M4 subsystem*, including its cores, on-chip RAM and ROM, cache, memory translation hardware, interrupt facilities, and system-bus interface. This is not interchangeable with the camera/ISP engines or the PowerVR graphics core. [1, 9]

The difference matters because a software example written for a "Ducati" chip family may assume a different core revision, memory map, memory region, or reset sequence.

1.3 The other main pieces

Term	Simple meaning	Job in our experiment
MPU / host	Cortex-A15 running Linux	Loads firmware and controls the IPU
IPU CPU0 / CPU1	Two Cortex-M4 cores	CPU0 runs the test; CPU1 remains held
PRCM / PRM	Power, reset and clock controls	Releases the shared IPU block, then CPU0
L2 SRAM	64 KiB of physically present IPU on-chip RAM	Holds the executable and its small runtime state
Unicache / SCACHE	Shared IPU cache system	Has an internal translation/attribute MMU
AMMU	The IPU-internal attribute MMU	Makes local address 0x00000000 reach SRAM
L2 MMU / IOMMU	The IPU's external-bus address translator	Needed for appropriately mapped access to DDR/peripherals
remoteproc	A Linux framework plus an OMAP-specific driver	Loads an ELF and starts/stops the remote processor
Device Tree	Hardware description Linux reads	Connects the IPU to SRAM, resets, clocks, IOMMU, mailbox
CMA / reserved memory	Ways Linux provides suitably managed RAM	Useful for larger DDR/IPC buffers, not our initial SRAM-only test

1.4 The component relationships



OMAP5432 host–IPU architecture diagram

Figure 1 — Conceptual functional paths. The host firmware loader, the internal AMMU, and the external IOMMU serve different purposes. Arrows are explanatory, not a complete representation of TI's interconnect topology.

The **Device Tree** tells Linux that an IPU exists and which hardware resources belong to it. **remoteproc** controls the remote processor. The OMAP-specific remoteproc driver performs the hardware-dependent actions: mapping internal SRAM, preparing the device and interacting with reset controls. **PRCM** controls whether the processors may run. **AMMU** determines part of the local address interpretation. The external **IOMMU** becomes important when IPU requests need to leave the local memory domain for the system bus. [1, 2, 9]

2. What "remoteproc driver" actually means

2.1 A framework and a platform driver—not firmware running on the M4

Linux has a generic **remoteproc framework** for processors controlled by another processor. It provides common concepts such as firmware files, loading ELF segments, resource tables, `start`, `stop`, and crash/recovery state. An OMAP-specific `omap_remoteproc` driver connects this framework to the OMAP IPU's actual reset, clock, SRAM, IOMMU and mailbox hardware. [6]

remoteproc is Linux host-side code. It is not an operating system on the Cortex-M4. The firmware is a separately compiled program for an ARMv7-M/Cortex-M4 target. The host controls the lifetime of that program.

Our initial Pyra kernel work ported an OMAP5-capable remoteproc implementation into the Pyra's Linux 5.6.19-based kernel, enabled the appropriate configuration, and introduced a board-specific IPU Device Tree configuration. Phase 9.3 then corrected a Linux reset polling path that could sleep in an atomic context; that fixed a host-side fault but did not, by itself, prove execution of valid instructions on the M4. [9]

2.2 Why one Linux remoteproc device represents two M4 CPUs

For this OMAP5 IPU configuration, Linux exposes **one remoteproc device for the IPU subsystem** (`55020000.ipu`), not one independently addressable remoteproc for each M4. This is consistent with the OMAP remoteproc Device Tree representation: a subsystem is the managed hardware unit, and the low-level driver controls its individual reset lines. [9]

In the final tested configuration the driver explicitly obtains individual CPU0 and CPU1 resets and releases only CPU0. Earlier behavior could deassert an array of both CPU reset lines at once. Holding CPU1 until there is firmware prepared to manage it reduces the number of uncontrolled execution paths. [4]

2.3 The ELF and resource table

The test firmware is an **ELF32 ARM executable**, not an ordinary Linux executable. ELF program headers tell the Linux loader which bytes to place at which **device addresses (DA)**. A special `.resource_table` section announces anything the firmware needs the host to prepare.

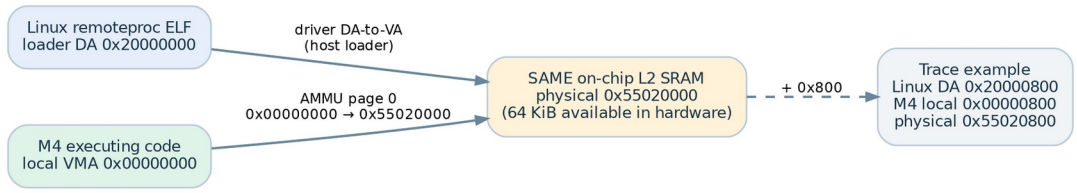
Common resource types include:

- **`RSC_TRACE`** — publish a buffer that Linux can read for diagnostic text.
- **`RSC_CARVEOUT`** — request a memory region, often allocated with a suitable backing area and mapped for the remote processor.
- **`RSC_DEVMEM`** — request an address mapping to an already existing device or physical resource.
- **`RSC_VDEV`** — advertise a virtual device, commonly including virtio/RPMsg communication rings.

Phase 9.12 uses **exactly one resource, `RSC_TRACE`**, pointing to an existing 512-byte buffer in internal SRAM. It requests **no** DEVMEM mappings, CARVEOUT allocations or VDEV/RPMsg rings. The resource table's metadata does not magically create a program start address or configure the internal AMMU. [3, 6]

3. The three address spaces you must not mix up

This is the most important conceptual trap in the OMAP5432 bring-up. The **same physical SRAM byte** is identified in three different ways, depending on who is looking at it.



Host loader DA, M4-local VMA, and physical SRAM address

Figure 2 — Two independent ways to reach the same on-chip RAM: host-side ELF loading and M4-local execution.

Address type	Example	Who uses it?	Meaning
M4-local execution address / VMA	0x00000000	Cortex-M4	Where the M4 expects its vector table and instructions
Linux remoteproc loading DA	0x20000000	Host loader	Device address recognized by the patched OMAP remoteproc SRAM mapping
Physical on-chip L2 SRAM address	0x55020000	OMAP interconnect / CPU host mapping	Hardware location of the 64-KiB SRAM block
AMMU translated base	0x55020000	IPU-local translation	Mapping target of local page 0

For example, the trace buffer is at **M4-local 0x00000800**, Linux firmware loading **DA 0x20000800**, and **physical SRAM 0x55020800**. They all designate the same intended trace bytes once the relevant translations are configured. [1, 3, 5]

The first conversion (**Linux DA to a host-accessible SRAM mapping**) belongs to the remoteproc loader/driver. The second conversion (**M4-local address to on-chip SRAM**) belongs to the internal AMMU. One does not automatically perform the other's job.

3.1 AMMU: translation inside the IPU

The **SCACHE_MMU_IPU**, commonly called the **AMMU**, is associated with the shared Unicache. It assigns address translations and access/cache attributes to IPU-local address regions. The TRM provides small pages of 4 or 16 KiB and also larger pages. For the tested firmware, the relevant page is page 0: the M4 sees local 0x00000000 and the translation must reach physical 0x55020000. [1]

On the tested kernel, the register readback **before** the Phase 9.11.5 write showed:

```
P0_ADDR   = 0x00000000
P0_XLTE   = 0x00000000
P0_POLICY = 0x0000000B
```

The successful phase changed the page-0 translation to **P0_XLTE = 0x55020000**, verified it, and only then permitted CPU0 release. The AMMU configuration space is at physical 0x55080000, with SMALL_XLTE_0 used by our patch at 0x550809A0. [2, 5]

Interesting discrepancy: The OMAP5432 TRM documents a page-0 reset translation value of 0x55020000, while our actual pre-start register readback showed 0x00000000. The records establish the discrepancy, but they do **not** establish exactly what previously changed that register (reset sequencing, firmware or another software path). Do not generalize our measured value into a universal silicon reset default. [1, 5]

3.2 IOMMU: translation when an IPU access leaves for the system bus

The **IPU L2 MMU**, represented in Linux as the OMAP **IOMMU**, translates IPU-originated requests toward the L3 interconnect—for example, external DDR or memory-mapped peripherals. The OMAP5 IPU IOMMU block is at approximately **0x55082000** in the tested Device Tree. It is **not** the same block as the AMMU registers at 0x55080000. [1, 9]

The practical distinction:

- **Local SRAM fetch:** The M4's local address is translated by the internal AMMU to the on-chip SRAM; our tiny test does not need an additional external IOMMU mapping for that instruction fetch.
- **External DDR/peripheral access:** The firmware address, internal AMMU policy, external IOMMU mapping, physical target, cache attributes and resource ownership must be mutually consistent. A valid page-0 AMMU mapping alone cannot authorize arbitrary DDR access.

TI's software manuals describe the Unicache MMU/AMMU as the first translation stage and the IOMMU as the later stage for off-chip resources. Not every IPU address necessarily passes through the same external path; the SRAM boot case is deliberately a separate, simple test. [1, 10]

Do not fix a local boot mapping by blindly adding an RSC_DEVMEM entry for address zero. That would configure the wrong part of the system and could introduce unexpected bus traffic.

3.3 Why the bogus pgd messages did not invalidate the test

The successful host kernel logged bogus pgd 0x0, da 0x0 and bogus pgd 0x0, da 0x20000000 **before** CPU0 was released. Those entries came in the diagnostic period when an IOMMU lookup was requested for addresses without a valid external mapping. Afterward the local AMMU commit succeeded, CPU0 was released, and the actual M4 SRAM markers appeared. [5]

Thus these messages were **not evidence that the M4 C firmware tried to use external DDR**. Conversely, success in local SRAM does **not** establish that a future IOMMU-backed DDR access will work. Read the timestamps, call path, and firmware resource table together.

4. What the 64-KiB SRAM can and cannot do

The IPU contains **64 KiB of physical L2 SRAM at 0x55020000–0x5502FFFF**. It is a small, fast, on-chip memory well suited to minimal boot and deterministic tasks. It is also much easier to validate than a DDR-based remote-processor setup. [1]

4.1 Good candidates for SRAM-only firmware

- First instruction-execution proof and short diagnostic messages.
- Bare-metal control loops, small state machines and simple mathematics.
- Tiny freestanding C applications with modest .text, .data, .bss and stack requirements.
- A small private command/status area with simple polling, once CPU-to-CPU synchronization has been designed.
- A local log buffer or a small non-DMA scratch buffer.

4.2 Cases likely to need more than SRAM

- Large RTOS images, large libraries, C++ runtimes, multiple task stacks or substantial heaps.
- Framebuffers, video/audio buffers, sizable captured sensor data or extensive logging.
- RPMsg/virtio vrings and message buffers shared with Linux, depending on the selected implementation.
- Larger DMA-capable buffers or shared memory needed by bus-master peripherals.
- Firmware with large code/data images or with significant simultaneous CPU0 and CPU1 workloads.

Important caveat: 64 KiB physically available does not mean **64 KiB already mapped at M4 address zero**. Our small-page mapping is configured for a **16-KiB local window**, and Phase 9.12 uses only the **first 4 KiB**. Using the rest of the physical SRAM may require additional AMMU pages or another validated mapping scheme. It also requires matching linker ranges, resource-table DAs, and host SRAM-loader support. Never assume the complete 64 KiB is accessible just because it is present. [1–3]

5. When is CMA or a dedicated memory pool needed?

CMA (Contiguous Memory Allocator) is a Linux facility for allocating physically contiguous memory when a device or driver needs it. A Device Tree **reserved-memory** region can designate memory for specific hardware, and a region described as a shared-dma-pool may be used by the relevant DMA/CMA plumbing. A dedicated CMA region is not the same thing as “the M4 has its own memory”: it is host-managed **external system RAM** with an allocation/ownership policy. [9, 11]

The original Pyra IPU Device Tree has no dedicated memory-region pool. Linux therefore prints a warning similar to “device does not have specific CMA pool.” **For the tested SRAM-only firmware, that warning is not a failure:** code, trace and stack remain inside IPU L2 SRAM and the resource table requests no extra pool. [3, 9]

Firmware design	Extra reserved/CMA memory needed?	Other essential work
4-KiB Phase 9.12 SRAM self-test	**No**	Correct SRAM loader, AMMU and reset sequence
Slightly larger program wholly in the 64-KiB L2 SRAM	**Not inherently**	Expand/verify M4-local mapping and linker layout
DDR-based code/data allocated dynamically by remoteproc	**Often**, depending on driver/allocation design	Allocate suitable backing memory and set IOMMU mappings
Shared Linux–M4 buffers or large data areas	**Often useful**, but not universally mandatory	Lifetime, cache maintenance, physical/IOVA mapping, synchronization
Virtio/RPMsg vrings and shared message buffers	**Typically need allocated shared RAM**	RSC_VDEV, vring addresses, mailbox notifications, IOMMU, ownership
Fixed-address peripheral MMIO	**Not CMA simply because it is MMIO**	Clocking, permissions, device ownership, valid AMMU/IOMMU mapping

A **static reserved carveout** can be appropriate when firmware requires a fixed memory location or the system must ensure Linux cannot allocate that memory for unrelated purposes. A **CMA pool** can be appropriate when allocations must be physically contiguous but can be made and released dynamically. With a capable IOMMU, some data buffers may be mapped from non-contiguous physical pages; actual remoteproc driver and DMA-device requirements determine what is possible. **Do not add a guessed "extra 64 MB CMA" parameter as a universal remedy.** It neither fixes a wrong Cortex-M4 reset vector nor corrects a missing AMMU page-0 translation. [6, 9–11]

For an eventual DDR-backed design, plan the physical reservation, remoteproc resource entries, IOMMU IOVAs, cacheability, Linux-side ownership and device DMA constraints **as one system**. A pool only solves the backing-memory part.

6. Correct and safe startup sequence

The OMAP5432 TRM specifies that the IPU cache/MMU block must come out of reset and be configured **before CPU0 is released**. The two Cortex-M4 CPU reset controls are separate. The tested sequence follows this principle, while still making an important distinction between documented procedure and the limited cache behavior we have actually exercised. [1, 2]



IPU PRM reset control sequence

Figure 3 — Simplified interpretation of RM_IPU_RSTCTRL *for a CPU0-only boot. This illustrates the observed path; it is not a substitute for the complete PRCM procedure.*

Reset control bit meaning for RM_IPU_RSTCTRL (physical 0x4AE06910): bit 2 holds the IPU MMU/cache interface in reset; bit 1 holds CPU1; bit 0 holds CPU0. A bit set to **1** means reset asserted. Therefore 0x7 means all three held, 0x3 means the MMU/cache reset is cleared while both CPUs remain held, and 0x2 means CPU0 is released while CPU1 is still held. [1]

A correct conceptual order is:

1. Boot Linux with the M4-enabled Device Tree. Confirm Linux recognizes ti,omap5-ipu and registers the IPU remoteproc device.
2. Use the correct kernel driver and firmware identity. The **working** phases add a CPU0-only path, a guarded AMMU configuration step and verified ELF loading.
3. Keep CPU0 and CPU1 in reset while preparing the subsystem. Bring the shared MMU/cache block out of reset in the documented PRCM sequence; ensure the required clocks are active.

4. Let remoteproc parse the ELF, allocate/validate declared resources and copy ELF LOAD segments into host-visible SRAM. Confirm that the loader's DA `0x20000000` resolves to physical L2 SRAM.
5. Check the first two vector words in L2 SRAM. For this particular firmware the expected **initial MSP is `0x00001000`** and the **Thumb Reset Handler address is `0x00000041`**.
6. Configure and validate AMMU page 0 before releasing CPU0. The tested patch sets translation `P0_XLTE` to `0x55020000`, reads it back, and verifies the reset state and surrounding registers.
7. Release **CPU0 only**, using the existing reset-controller path. Keep CPU1 asserted. Avoid inventing a second deassert operation in the firmware launcher.
8. Observe firmware-produced markers and changing data. Merely seeing `remote processor ... is now up` proves the driver completed its start call, **not** that the M4 executed a valid program.

What remains unproved: The Phase 9.11.5 helper did not demonstrate every TI internal cache-initialization operation for arbitrary workloads. Our SRAM-only non-IPC firmware worked empirically, but later cacheable DDR/sharing features require additional verification. [2, 3]

6.1 How our kernel phases contributed

Stage	What changed	What it demonstrated
Early Pyra-M4 project	OMAP5 remoteproc driver, IPU DT, kernel configuration	Linux can see and manage the IPU
Phase 9.3	Atomic-safe PRM reset polling	Addressed BUG: scheduling while atomic; not yet an M4 execution proof
Phase 9.7	Separate CPU0-only reset control	CPU1 can remain in reset during a CPU0 test
Phases 9.8–9.11.4	IOMMU, PRM, AMMU readbacks and staged tests	Identified loader/translation/reset state and safe register programming
Phase 9.11.5	Guarded AMMU page-0 translation + ordered CPU0 release	v5 firmware wrote M5A1, M5B2, M5D3
Phase 9.12	C runtime and SRAM self-test firmware; no further kernel patch	M12A/I/B/C, OK=1, increasing heartbeat

This history matters: **most intermediate patches were diagnostics or safeguards**, while the tested page-0 AMMU setting and CPU0 reset handling are functional components of our successful boot path. That does not prove that *every* unsuccessful earlier run had the same root cause. [2–5, 9]

7. Why entry points and memory layout are difficult

A Cortex-M4 does not start like a Linux process. There is no Linux `execve()`, user stack or dynamic linker. At reset, the ARMv7-M core uses a **vector table**: the first word provides its initial main stack pointer (MSP) and the second word identifies the Reset Handler. Addresses and instruction encoding must agree with Cortex-M4's **Thumb** execution model.

Our vectors are:

```
M4 local address   Value              Meaning
0x00000000         0x00001000         Initial MSP (top of stack)
0x00000004         0x00000041         Reset_Handler address with Thumb bit
0x00000040         Thumb instructions Actual Reset_Handler code
```

Why `0x41` and not `0x40`? Bit 0 communicates Thumb state. The actual instruction bytes begin at the aligned address `0x40`; the reset vector and ELF entry value encode `0x41`. A mismatched vector or an ARM/A-profile executable cannot safely substitute for Cortex-M4 code.

Why the ELF entry alone is insufficient: On this IPU the observed driver path has no separate programmed boot-address register (bootreg=no). Correctly placing the vector table at the M4-local zero address is the critical hardware startup contract. The ELF e_entry of 0x41 matches and helps with validation, but writing that metadata does not itself remap address zero. [2, 3, 5]

7.1 VMA, LMA and ELF program-header address

Two linker concepts must be distinguished:

- **VMA (virtual memory address):** the address that the compiled instructions use when they execute on the M4, e.g. .text at local 0x00000040 or runtime .data at 0x00000680.
- **LMA (load memory address) / firmware DA in this project:** the address at which Linux places bytes, e.g. 0x20000040 for .text, or 0x20000600 for .data initializers.

The ELF's **PT_LOAD program headers** matter to remoteproc. For Phase 9.12, readelf -l shows eight LOAD segments plus GNU_STACK. The .vectors segment has virtual address 0x0 and load address 0x20000000; .text starts at virtual 0x40 and load address 0x20000040. A unique detail is .data: its initial bytes are **loaded at 0x20000600** but the running C variables reside at **local 0x680**. Startup assembly must copy those eight bytes. [3]

A generic linker script placing .text at 0x55020000, or one using 0x20000000 as the M4 program counter without a corresponding map, would not match our proven boot setup.

7.2 Why the resource table is elsewhere

The ELF .resource_table is located at **DA 0x20001000**, outside the tiny 4-KiB execution window. It is **host-side metadata** parsed by remoteproc; the firmware does not branch to it. Keeping that metadata separate prevents it from colliding with the vector table, stack or trace. It does not mean the Cortex-M4 is using DDR. [3]

8. The Phase 9.12 Cortex-M4 firmware project, file by file

The actual project is named **Pyra-M4-Phase-9.12-SRAM-Runtime-Validation**. It includes the firmware sources, ELF, map file, validation tools, a guarded host launcher and an archived Phase 9.11.5 reference package. The new firmware did not introduce a new kernel, Device Tree or AMMU patch; it reused the successfully tested CPU0 boot path. [3]

8.1 Directory guide

Project path	Purpose
firmware-v12/firmware/startup.S	M4 vector table, low-level reset handler, .data copy, .bss clear, fault marker
firmware-v12/firmware/linker.ld	Exact memory addresses, load locations, region size assertions
firmware-v12/firmware/main.c	Freestanding C runtime checks, scratch SRAM test, heartbeat loop
firmware-v12/firmware/resource.c	One Linux remoteproc RSC_TRACE resource, trace and scratch buffers
firmware-v12/Makefile	Cross-compiles ARMv7E-M/Thumb firmware with an embedded toolchain
firmware-v12/build/*.elf	Checked firmware image loaded by remoteproc
firmware-v12/build/*.map	Linker report: section positions, sizes and symbols
firmware-v12/tools/check_elf.py	Rejects an ELF that breaks our strict SRAM/entry/resource contract
firmware-v12/tools/check_trace.py	Checks trace markers and a heartbeat that actually advances
firmware-v12/run-pyra.sh	Verify, probe, install, one-shot start, status and stop operations
start-phase912.sh	Explicitly confirmed start wrapper and kernel log capture

Project path	Purpose
tools/collect-phase912.sh	Saves logs, flags, firmware identity and four trace samples
phase9115-reference/	Preserves the previously working host kernel implementation for audit

8.2 The deliberate 4-KiB internal layout

Every M4 runtime reference is confined to the first **4 KiB** of the 64-KiB physical SRAM. These are **local M4** addresses; Linux loads through DA 0x20000000 + local offset where applicable.

M4-local range	Role	Notes
0x0000–0x003F	Vector table	MSP 0x1000, Reset PC 0x41
0x0040–0x05FF	Code / read-only constants	0x600 is the reserved end limit; actual .text smaller
0x0600–0x067F	.data initializer storage	Linux places initializer bytes here
0x0680–0x06FF	Runtime .data	Startup copies values into this range
0x0700–0x07FF	.bss	Startup zeroes the used words
0x0800–0x09FF	Trace	512 bytes, RSC_TRACE DA 0x20000800
0x0A00–0x0BFF	Scratch	128 volatile 32-bit words / 512 bytes
0x0C00–0x0FFF	Stack	1-KiB stack; MSP begins at exclusive top 0x1000
Host metadata 0x20001000	.resource_table	Not executed or dereferenced by M4

A region's reserved upper bound is **not** the size of the currently populated ELF section. For example, the supplied linker map shows only **eight bytes** of .data and **eight bytes** of .bss, while the layout reserves larger areas so the regions cannot overlap. [3]

8.3 startup.S: the first instructions that matter

At local address zero, startup.S installs the required vector table. Its handler starts at local 0x40 and emits M12A directly into SRAM **before using the C stack or globals**. That isolates a failure to fetch instructions from a later C initialization problem.

Next, simple assembly loops copy the initialized .data words from **local 0x600** to **local 0x680** and clear the .bss words around **0x700**. The handler then emits M12I and branches to main(). It does not depend on libc startup code, DDR, malloc, semihosting or a system timer. An exception path uses M12F/fault text when possible and stops in a low-level wait loop. [3]

Proof limitation: On some ELF-loading paths, zero-filled (NOBITS) memory may already have been cleared by Linux. Our startup *does* execute the .bss clearing loop, and the C test finds zeros, but that alone cannot prove the buffer was nonzero immediately before that loop. The .data copy test is stronger because the initializer bytes are deliberately loaded at a different address from the runtime variables. [3]

8.4 linker.ld: the contract between the two processors

The linker script must coordinate two viewpoints: local M4 addresses and the Linux loading addresses. It establishes ENTRY(Reset_Handler) but also preserves a real vector table at zero. It separately places a copyable .data initializer, a runtime .data section, the .bss, trace, scratch, and a fixed stack.

The linker contains **assertions** for the code-size limit, .data and .bss sizes, exact trace and scratch sizes, and stack top. That means an innocent-looking future code change which makes the firmware too large can fail at link time rather than silently overwriting the trace or stack. [3]

A developer should inspect **readelf -h**, **readelf -l**, and the .map file after *every* linker or toolchain change. For this project, the expected ELF is **32-bit little-endian ARM EABI**, entry **0x41**, Cortex-M4 Thumb instructions, and no externally allocated code/data LOAD segments.

8.5 main.c: a useful but intentionally small C runtime

The compiled C program first checks that a local stack variable really resides inside the reserved stack (0xC00–0xFFF). It confirms two initialized global variables (0x13579BDF, 0x2468ACE0) survived the startup copy and confirms zero values for its .bss globals. It writes an additional marker into .bss and reads it back.

It then fills **128 volatile 32-bit scratch words** at local 0xA00–0xBFF with a varying test pattern and reads each word back. On success it writes M12C, OK=1, and a readable hex heartbeat. On error it emits an ERR= code (where possible) and stops rather than pretending to succeed. [3]

The heartbeat is a **busy-wait loop**, not a hardware timer. This is deliberate: it proves repeated execution without requiring interrupt setup, DMTimer clocks, DMA or MMIO. **Its numerical rate is not a performance benchmark**. The ASCII hex digits update one byte at a time, so an occasional in-flight host read could catch an incomplete value; that should be treated as inconclusive and logged. [3]

8.6 resource.c: exactly what the host needs to know

resource.c defines one RSC_TRACE entry: type=2, DA 0x20000800, length **512 bytes**, name pyra_m4_v12_sram. The trace is a real SRAM buffer, not a claim that remoteproc should allocate external memory. The table has no VDEV, no DEVMEM, and no CARVEOUT entries. [3]

This simplicity was intentional. If we add a new resource type later, both the kernel's resource handling and the M4's actual address map must be reconsidered; adding a resource entry is not equivalent to adding instructions to main.c.

8.7 Cross-compilation and binary identity

The firmware uses -mcpu=cortex-m4 -mthumb, a **freestanding embedded toolchain**, and no standard libraries. It can be built with arm-none-eabi-gcc or with clang --target=armv7em-none-eabi plus LLD. **Do not compile it with the Pyra's normal arm-linux-gnueabihf-gcc application flags** such as -mcpu=cortex-a15 or an ordinary Linux runtime/linker script: Cortex-A15/Linux and Cortex-M4/bare metal are different targets.

The supplied ELF has SHA-256:

```
1c8b2c16aa02a7231d810a3072d11954c7f319ab2706859afde050e955c17a89
```

The default launcher pins this exact binary identity. A locally rebuilt ELF can differ legitimately because of toolchain version, alignment, section placement or code generation. When that happens, rerun the ELF/static tests and explicitly review the differences rather than deleting the hash check. [3]

9. Preparing and performing the first firmware experiment

This section documents the **known-good Phase 9.11.5 + Phase 9.12 path**, not a recipe for blindly running the firmware on an older kernel. The original Phase 9.3 kernel lacks subsequent CPU0-only and AMMU safeguards; a direct start there is a different, more hazardous experiment. [2–4]

9.1 Safety and environment checklist

Before starting:

1. Keep a separately bootable **m4base recovery kernel**, with its **own kernel-image file** and original non-M4 DTB. Merely having two boot entries pointing to one vmlinuz is not an independent recovery path.
2. Boot the known-good **m4ipu** kernel and the M4-enabled Device Tree. The release string alone does not prove kernel identity, since multiple builds shared 5.6.19-pyradef-m4test.
3. Verify /sys/class/remoteproc/remoteprocN/name identifies 55020000.ipu. Find by name, not by blindly assuming it must always be remoteproc0.
4. Confirm the IPU is offline and has **not already been started in this Linux boot**. In our 9.11.5 implementation one attempt consumes a kernel-side latch.
5. Check that the CPU0-only flag and Phase-9.11.5 boot flag are active, while older repair/staging modes are inactive. Confirm the fixed firmware contracts before changing anything.
6. Preserve the original successful v5 ELF under its own name. The v12 firmware uses pyra-m4-sram-v12.elf and does not overwrite the older one.
7. Keep a terminal/log collection method ready; do not initiate the hardware test unless you can reboot into recovery if Linux hangs.

9.2 Read-only checks and a single authorized start

From the **Phase 9.12 project root**, first run the no-root ELF validation and test suite:

```
python3 firmware-v12/tools/check_elf.py \
  firmware-v12/build/pyra-m4-sram-v12.elf
python3 -m unittest discover -s firmware-v12/tests -v
sha256sum -c SHA256SUMS
./firmware-v12/run-pyra.sh --verify
```

After confirming you booted the correct m4ipu entry, perform the project's guarded hardware probe:

```
cat /sys/class/remoteproc/remoteproc0/state  # expect offline on this Pyra
cat /sys/module/omap_remoteproc/parameters/pyra15_boot_once
sudo ./firmware-v12/run-pyra.sh --probe
```

If the verification fails, **stop**. Do not force the script to skip the missing flag. The **only** intended start command for the tested 9.11.5 environment is:

```
sudo ./start-phase912.sh BOOT-M4-SRAM-ONCE
```

This wrapper calls the safeguarded launcher, validates the supplied firmware, installs the v12 filename only, disables automatic remoteproc recovery, sets the firmware, and performs exactly **one** hardware start. It subsequently verifies changing heartbeat samples and logs relevant kernel messages. It does **not** replace the kernel image or rewrite PRM/AMMU registers directly from a shell. [3]

9.3 Collect evidence without corrupting the experiment

When the Pyra remains responsive, collect the runtime state and trace:

```
sudo ./tools/collect-phase912.sh \
  /home/USER/pyra-phase912-evidence
sudo ./firmware-v12/run-pyra.sh --status
```

The collection script writes diagnostic files into the indicated directory. If that directory becomes root-owned, restore ownership afterward from your normal user account; do not loosen all permissions indiscriminately:

```
sudo chown -R "$(id -un):$(id -gn)" \
  /home/USER/pyra-phase912-evidence
```

Do not start again during the same boot, even after a normal stop. If a hardware fault appears, preserve the logs and boot via `m4base` before proceeding.

9.4 What counts as a successful result?

The expected trace begins with:

```
M12A
M12I
M12B
M12C
HB=XXXXXXXX
OK=1
```

A verified result needs **all four markers, `OK=1`, and increasing heartbeat values across multiple spaced reads**. A single still frame, an ELF accepted by remoteproc, or `state=running` alone is not enough.

The supplied Pyra evidence reported these values:

Sample	Hexadecimal	Decimal	Change from previous
1	0x02D1	721	—
2	0x02DC	732	+11
3	0x02E6	742	+10
4	0x02F1	753	+11

Those values establish continued M4 execution between host reads. They do **not** establish an exact M4 clock speed, a validated interrupt subsystem, external DDR access, or safe cache coherence with Linux. [5]

10. Why repeated starts are dangerous

On a conventional Linux application, restarting often means a clean new process. A remote Cortex-M4 is not a normal process. It has **persistent hardware state**: reset lines, running/parked cores, cache contents, address translations, device state and potentially active bus requests. A remoteproc `stop`, `offline`, or `crashed` indication does not prove that every IPU submodule has returned to the precise power-on state described in the TRM.

Potential hazards include:

- **Uncontrolled second-core execution:** a driver that releases both CPU resets may start CPU1 without suitable vectors/stack/program organization.
- **Stale AMMU or cache state:** new firmware may execute under an old translation or attribute configuration.
- **Partially torn-down IOMMU mappings:** a stale firmware access may target memory no longer owned by the IPU.
- **Automatic recovery loops:** repeated start, fault and recovery can obscure the first fault, overwrite trace, or leave the system less stable.

- **DMA/MMIO still in flight:** a remote processor or peripheral could modify system memory or registers after the host assumes it has stopped.
- **Confusing reset status with actual reset:** RM_IPU_RSTST reports events; its bits are not the same as the asserted reset-control bits and may be write-one-to-clear. Do not clear status registers casually.
- **L3 bus faults or system hangs:** a bad bus address can affect more than the M4 and could require an external power cycle/recovery boot.

For these reasons the experimental 9.11.5 kernel refuses an additional in-boot AMMU/CPU0 commit. The Phase 9.12 launcher disables remoteproc auto-recovery and requires an explicit one-shot authorization token. This is not an inconvenience to work around: it is an important part of the experiment's validity. [2, 3]

Safe practice: One controlled attempt per fresh boot. On success, capture evidence and stop only through the known-good helper if desired. On fault, hang or unknown state, avoid manual devmem reset writes, do not spam echo start, and return to the independent m4base recovery configuration. A reboot is safer than assuming the hardware has reset; a power cycle may be required in some failure modes. Avoid modifying the proven kernel and firmware simultaneously.

11. Known limits and the right order for future work

The v12 proof is impressive because it is deliberately small. It **does not** test external IOMMU mappings, DDR code/data, cache coherence, timers, interrupts, DMA, external peripheral MMIO, mailboxes or RPMsg. It also does not demonstrate normal exception recovery, secure multi-core handover, sustained low-power operation, or full two-core operation. [3, 5]

An incremental roadmap minimizes ambiguous failures:

1. **Freeze the reference:** archive the current working kernel image, active DTB, pyra-m4-sram-v12.elf, hashes, logs and startup parameters.
2. **SRAM-only command/status:** add a tiny host/M4 state machine in dedicated local SRAM, with sequencing and timeouts. Preserve current AMMU and reset behavior. Validate cache visibility and access ownership before calling it IPC.
3. **Controlled interrupts or mailbox:** introduce one event source at a time, measuring both successful service and failure paths. Do not assume a mailbox is already being used merely because Device Tree declares one.
4. **External memory mapping:** introduce one pre-agreed DDR range and matching IOMMU translation, with a clear allocator or reserved memory plan and cache-maintenance rules.
5. **Virtio/RPMsg:** introduce RSC_VDEV, vrings and mailbox notifications only after allocations and mapping have been verified, and test clean teardown/recovery separately.
6. **CPU1 and larger runtimes:** design reset release, vectors, stack, shared cache arbitration and memory ownership explicitly before enabling the second M4.

The best future optimization is not to delete safety checks immediately. It is to **isolate the smallest verified functional kernel path**, remove obsolete diagnostic probes only after a successful regression test with the unchanged v12 firmware, and keep a way to recover from a failed boot.

12. Troubleshooting guide

Symptom	Likely interpretation	Recommended response
No /sys/class/remoteproc IPU device	Wrong DTB, driver configuration, clock/device probe failure	Verify m4ipu DT and kernel; do not start
pyra15_boot_once missing	Older/incorrect running kernel image	Verify active kernel identity; avoid bypassing launcher
BUG: scheduling while atomic	Reset polling in unsuitable context	Verify Phase 9.3 PRM fix is present; inspect full trace

Symptom	Likely interpretation	Recommended response
ELF rejected before start	Vectors, segment DAs, section placement or resource contract mismatch	Inspect ELF/map; do not alter kernel safety checks to force it
P0_XLTE readback unexpected	IPU internal mapping not in recognized state	Abort and inspect reset/configuration history
CPU0 still held after attempted start	Reset sequence failed or safety gate rejected	Inspect PRM and driver logs; do not issue manual reset writes
running but no M12A	Host started device, but M4 instruction execution unproven	Check vector placement, AMMU, CPU0 reset; preserve logs
M12A, no M12I	Failure during assembly startup or memory init	Check .data/.bss layout and local access
M12I, no M12B	Problem handing off to C or stack	Verify compiled target, stack region and ABI
M12F, ERR=	Firmware detected a test failure or exception	Preserve exact code, no in-boot retry
All markers, heartbeat constant	Firmware reached tests but ongoing loop not established	Verify spaced samples and reset/fault state
bogus pgd before CPU release	May be unmatched *diagnostic* IOMMU translation lookup	Check timestamps and call path; not automatically an M4 DDR fault
Early L3 custom error in display bring-up	Separate Linux device/bus issue may exist	Examine stack and timestamps before attributing to IPU
No dedicated CMA pool warning	No special DDR pool associated with IPU	Not fatal for this SRAM-only test

13. Quick-reference checklist

Before touching hardware

- ☐ Verified independent `m4base` recovery image and correct `m4ipu` DTB.
- ☐ Correct active 9.11.5 kernel/driver flags, not merely the same `uname -r`.
- ☐ `55020000.ipu` exists, remoteproc is offline, and no earlier attempt occurred this boot.
- ☐ Exactly the expected firmware ELF hash, entry, vectors and resource table.
- ☐ Linux loader DA maps to L2 SRAM; M4-local page 0 is prepared before reset release.
- ☐ CPU0-only arrangement is intentional; CPU1 remains reset.
- ☐ Remoteproc automatic recovery is disabled before the experimental start.
- ☐ A log and rescue plan exist; no plan to manually force MMU/PRM register writes.

To claim success

- ☐ Kernel reports a permitted, verified AMMU commit and CPU0-only release.
- ☐ M4 itself produces `M12A`, `M12I`, `M12B`, `M12C` and `OK=1`.
- ☐ Four separated samples show increasing `HB=` values.
- ☐ No time-correlated M4 IOMMU/L3 bus fault or system hang invalidates the run.
- ☐ Kernel image, firmware SHA, trace and dmesg are archived for reproduction.

Appendix A. Glossary

Term	One-line definition
AMP	Asymmetric multiprocessing: different processors run different software under separate control.
AMMU	Attribute MMU close to the IPU shared cache; applies local translations and attributes.

Term	One-line definition
CMA	Contiguous Memory Allocator: supplies suitably contiguous pages from Linux-managed RAM.
DA	Device address: the address the Linux remoteproc loader uses for a remote segment.
Ducati	TI software/family name for the dual Cortex-M subsystem; not an extra chip.
ELF	Executable and Linkable Format, including segments, sections and entry metadata.
IOMMU	I/O memory-management unit for accesses from a device/remote processor toward a system bus.
LMA	Load memory address: where section bytes are loaded before runtime use.
MPU	Main Cortex-A15 host processor subsystem running Linux on the Pyra.
MSP	Main Stack Pointer used by the Cortex-M4 at reset.
PRCM	Power, reset and clock management block.
RSC_TRACE	Firmware resource-table entry advertising a host-readable trace buffer.
Remoteproc	Linux framework for loading and managing remote processor firmware.
SRAM	On-chip static memory; here the IPU 64-KiB L2 RAM.
Unicache	Shared cache subsystem used by the IPU cores.
VMA	Runtime address used by executable code and data.
VDEV / RPMsg	Virtio remote device and associated messaging facilities, not present in v12.

Appendix B. Evidence and references

Project sources and measured evidence (specific to this DragonBox Pyra):

1. **TI OMAP543x Technical Reference Manual**, SWPU249AG (December 2017). Supplied chapter excerpts: *Memory Mapping* (§2.4, p. 320), *Power, Reset and Clock Management* (§3.5.6.10, p. 396; table 3-423, p. 596), and *Dual Cortex-M4 IPU Subsystem* (§7.2–7.4, especially pp. 1416–1428). Hardware-level reference; note the difference between documented reset values and this test's observed pre-boot AMMU values.
2. **Pyra-M4-Phase-9.11.5-r2-Active-Kernel-Verification** project archive: kernel/pyra15-boot-helper.cfrag, safety flags, reset/AMMU registers, readback/rollback logic, single-attempt gating.
3. **Pyra-M4-Phase-9.12-SRAM-Runtime-Validation** archive: firmware/startup.S, linker.ld, main.c, resource.c; Makefile; check_elf.py, check_trace.py, run-pyra.sh, start-phase912.sh, README and linker map. All layout values here were compared with the packaged ELF and map file.
4. **Pyra-M4-Phase-9.7-CPU0-only** project archive: separate CPU0/CPU1 reset controls and post-start status checks.
5. **Pyra Phase 9.11.5 and 9.12 hardware evidence**, 24 October 2026: remoteproc/PRM/AMMU dmesg extracts, v5 M5A1/M5B2/M5D3, v12 M12A/I/B/C, OK=1, four progressive HB= trace samples and firmware SHA-256.
6. **Linux kernel remoteproc documentation**, *Remote Processor Framework*: https://www.kernel.org/doc/html/v6.6/staging/remoteproc.html (generic framework explanation; the Pyra uses an older, modified kernel).
7. **Linux remoteproc resource-table definitions**, include/linux/remoteproc.h: https://code.googlesource.com/linux/torvalds/linux/+/11e3b725cfc282efe9d4a354153e99d86a16af08/include/linux/remoteproc.h (resource types; exact kernel-specific behavior must be checked locally).

8. **TI SYS/BIOS documentation**, ti.sysbios.family.arm.ducati.Core: https://software-dl.ti.com/dsps/dsps_public_sw/sdo_sb/targetcontent/sysbios/6_75_01_05/exports/bios_6_75_01_05/docs/cdoc/ti/sysbios/family/arm/ducati/Core.html (historical use of Ducati).
9. **TI OMAP remoteproc Device Tree binding**: https://git.ti.com/cgit/ti-linux-kernel/ti-linux-kernel-next/tree/Documentation/devicetree/bindings/remoteproc/ti%2Comap-remoteproc.txt?id=300a3f7bcfbf9614ac85dbf5f11cec7a17dd879e (IPU resources, optional reserved memory and two-core subsystem abstraction). Also see Pyra_Kernel_M4_Complete_Change_Report_EN.docx for the original project baseline and Phase 9.3 PRM fix.
10. **TI Processor SDK IPC documentation**: https://software-dl.ti.com/processor-sdk-linux/esd/docs/05_03_00_07/linux/Foundational_Components_IPC.html (separate AMMU and IOMMU translations; examples are from other TI/SDK configurations and must not be copied into OMAP5432 unmodified).
11. **TI Processor SDK RTOS memory/resource guide**: https://software-dl.ti.com/processor-sdk-rtos/esd/docs/06_00_00_07/rtos/index_Foundational_Components.html (carveouts, vrings, CMA and shared-memory design; not features validated in the v12 SRAM test).

Verification scope: This guide reconciles the supplied Pyra project sources, the provided OMAP5432 TRM chapters, the captured successful logs, and relevant Linux/TI documentation. Where hardware behavior has not been exercised—especially DDR, system-bus IOMMU mapping, coherent caches and two-core execution—it is explicitly marked as unverified. The guide does not claim an exhaustive hardware reset validation or a production-certified firmware lifecycle.